

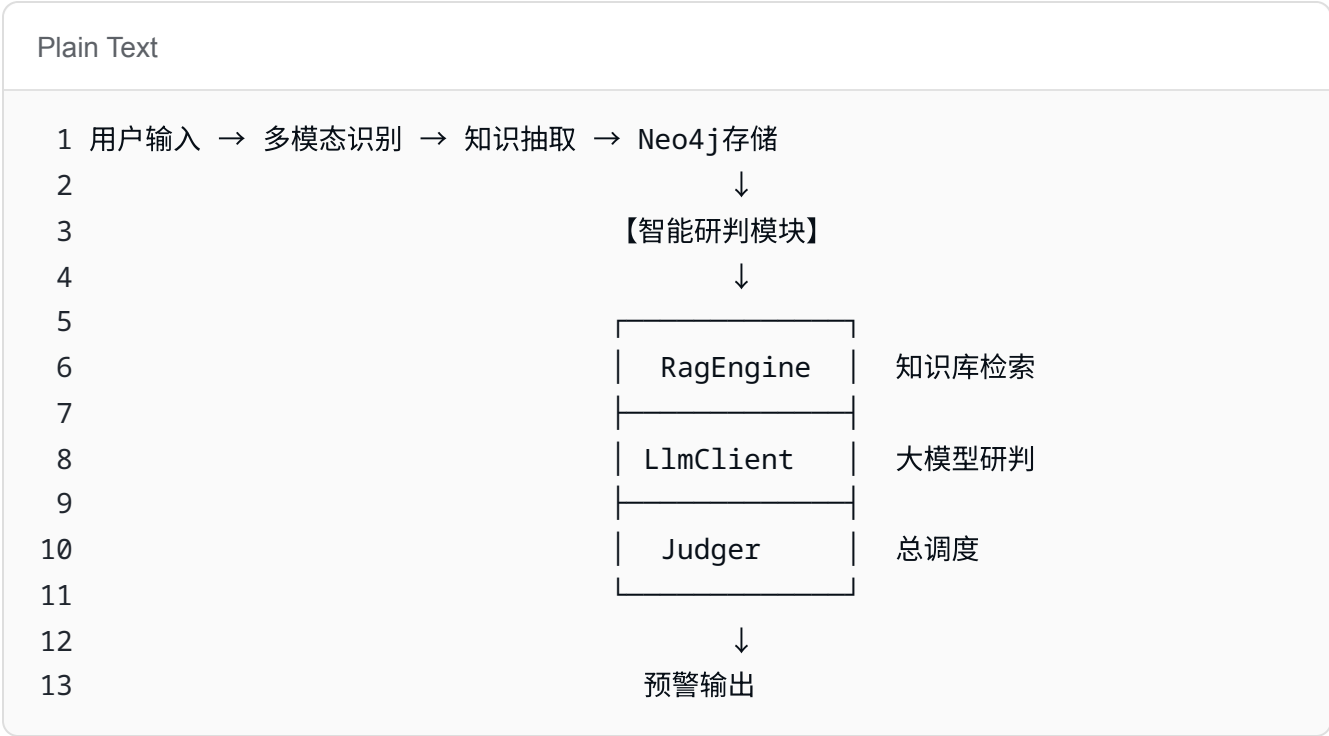
智能研判系统 - 接口设计

1.模块概览

1.1.模块定位

智能研判模块是整个 FraudShield 系统的**决策核心**。它接收多模态识别和知识抽取模块的输出，通过 **RAG（检索增强生成）** 技术，从历史诈骗案例库中检索相似案例，然后调用大模型进行综合研判，最终输出诈骗判断结果和预警建议。

1.2.模块位置



1.3.依赖关系

依赖方	输入	说明
上游（C 模块）	neo4j_data （格式 1.4）	从 Neo4j 查询的结构化数据
上游（前端 / 用户）	victim_info （文本）	直接文本输入
下游	研判结果（格式 1.5）	给预警模块 / 前端展示
外部依赖	硅基流动 API	大模型调用 + Embedding
外部依赖	ChromaDB	向量存储和检索

1.4.文件结构

Plain Text

```
1 JudgeCode/  
2 |— __init__.py          # 模块导出  
3 |— RagEngine.py         # RAG检索（向量库+知识库管理）  
4 |— LlmClient.py         # 大模型调用  
5 |— Judger.py            # 总调度  
6 |— config.py            # 配置（或共用项目根目录的config）
```

2.类 RAGEngine

2.1. 类说明

功能

管理知识库的加载，向量化存储和相似案例检索。

文件

RagEngine.py

2.2. 构造函数

```
__init__(persist_dir: str, collection_name: str)
```

项目	内容
功能	初始化 RAG 引擎，连接 ChromaDB 向量数据库
参数	
<code>persist_dir</code>	<code>str</code> ，必填，向量数据库持久化存储路径
<code>collection_name</code>	<code>str</code> ，必填，集合名称（如 <code>"fraud_cases"</code> ）
返回值	无
异常	如果 ChromaDB 连接失败，抛出异常

Python

```
1 from RagEngine import RAGEngine
2
3 engine = RAGEngine(
4     persist_dir="./data/vector_store",
5     collection_name="fraud_cases"
6 )
```

2.3. 方法

```
def _init_vector_store(self)
```

项目	内容
功能	从文本文件加载知识库（自动按"案例"切分）
参数	
<code>file_path</code>	<code>str</code> ，必填，知识库文件路径
返回值	<code>int</code> ，加载的案例数量

`load_from_json_file(file_path: str) -> int`

项目	内容
功能	从 JSON 文件加载知识库（自动按"案例"切分）
参数	
<code>file_path</code>	<code>str</code> ，必填，知识库文件路径
返回值	<code>int</code> ，加载的案例数量

`add_documents(documents: List[str], metadatas: List[Dict] = None) -> None`

项目	内容
功能	将文档列表存入向量数据库（自动向量化）
参数	
<code>documents</code>	<code>List[str]</code> ，必填，文档文本列表
<code>metadatas</code>	<code>List[Dict]</code> ，可选，每个文档的元数据
返回值	无

Python

```
1 docs = ["案例1：刷单诈骗...", "案例2：冒充公检法..."]
2 metadatas = [{"source": "fraud_cases.txt"}, {"source": "fraud_cases.txt"}]
3 engine.add_documents(docs, metadatas)
```

search(query: str, top_k: int = 3) -> List[Dict]

项目	内容
功能	检索与查询最相似的文档
参数	
query	str，必填，查询文本
top_k	int，可选，默认 3，返回结果数量
返回值	List[Dict]，每个元素包含 content（原文）和 score（距离，越小越相似）

Python

```
1 results = engine.search("我在网上兼职刷单，垫付了2万后提现不了", top_k=3)
2 for r in results:
3     print(f"{r['content'][:50]}... (相似度: {r['score']:.3f})")
```

返回格式

Python

```
1 [
2     {"content": "案例1：刷单诈骗...", "score": 0.679},
3     {"content": "案例2：冒充公检法...", "score": 0.838}
4 ]
```

```
search_batch(queries: List[str], top_k: int = 3) ->
List[List[Dict]]
```

项目	内容
功能	批量检索多个查询（一次API调用，更高效）
参数	
queries	List[str]，必填，查询文本列表
top_k	int，可选，默认 3，每个查询返回结果数量
返回值	List[List[Dict]]，每个查询对应一个结果列表

Python

```
1 queries = ["刷单兼职", "冒充公检法"]
2 results = engine.search_batch(queries, top_k=2)
3 # results[0] 是 "刷单兼职" 的结果
```

返回格式

Python

```
1 [
2     [{"content": "...", "score": 0.5}, {"content": "...", "score": 0.
7}], # 查询1
3     [{"content": "...", "score": 0.6}, {"content": "...", "score": 0.
8}] # 查询2
4 ]
```

3.类LLMCilent

3.1.类说明

功能

封装硅基流动大模型 API 调用，输出研判结果

文件

```
LlmClient.py
```

构造函数

```
__init__()
```

项目	内容
功能	初始化大模型客户端，从 <code>.env</code> 读取 API 密钥和配置
参数	无
返回值	无
异常	如果 <code>SILICONFLOW_API_KEY</code> 未配置，打印警告但不中断

方法

```
judge(victim_info: str, similar_cases: list, system_prompt: str = None) -> str
```

项目	内容
功能	调用大模型进行诈骗研判
参数	
<code>victim_info</code>	<code>str</code> ，必填，受害者描述文本
<code>similar_cases</code>	<code>list</code> ，必填，RAG 检索到的相似案例列表（格式同 <code>RAGEngine.search()</code> 返回值）
<code>system_prompt</code>	<code>str</code> ，可选，自定义系统提示词
返回值	<code>str</code> ，大模型返回的研判文本

4.类 FraudJudger

4.1.类说明

功能：智能研判总调度，整合 RAG 检索和大模型研判。

文件：`FraudJudger.py`

4.2.构造函数

`init()`

项目	内容
功能	初始化研判器，创建 RAG 引擎和大模型客户端
参数	无（所有配置从 <code>config.py</code> 或 <code>.env</code> 读取）
返回值	无

4.3.方法

`load_knowledge_base(file_path: str = None) -> int`

项目	内容
功能	加载外挂知识库（调用 RAGEngine）
参数	
<code>file_path</code>	<code>str</code> ，可选，知识库文件路径，不传则使用默认路径
返回值	<code>int</code> ，加载的案例数量

`judge(victim_info: str, top_k: int = 3) -> dict`

项目	内容
功能	对受害者信息进行研判（直接文本输入）
参数	
<code>victim_info</code>	<code>str</code> ，必填，受害者描述文本
<code>top_k</code>	<code>int</code> ，可选，默认 <code>3</code> ，检索相似案例数量
返回值	<code>dict</code> ，研判结果（格式见《数据结构设计文档》1.5 节）

`judge_with_neo4j_data(neo4j_data: dict, chat_history: str = None) -> dict`

项目	内容
功能	从 Neo4j 结构化数据进行研判（供 C 模块调用）
参数	
neo4j_data	dict，必填，从 Neo4j 查询的数据（格式见《数据结构设计文档》1.4 节）
chat_history	str，可选，原始聊天记录
返回值	dict，研判结果（格式同 judge()）